Chuleta DOM y Eventos

Desarrollo Web en Entorno Cliente · 2º DAW · Unidad 4 · Boletín DOM (1–4), Boletín Eventos (1–15) , Ejercicio Alumnos y Ejercicio Final TiendaDAW · incluye los conceptos que pide el profesor

Índice

★ Lo que pide el profesor (repaso rápido)
- DOMContentLoaded
- Delegación de eventos (plantilla)
- Arrays: insertar, eliminar, buscar
- Clases: crear e instanciar
- Funciones de string
- Manejo de HTML con template literals
- Atributos data-

0. Método para empezar cualquier ejercicio

1. Seleccionar elementos
- Colecciones vivas vs estáticas

2. Leer y modificar elementos

3. Crear, mover y borrar elementos

4. Eventos
- Registrar y quitar
- Tipos de evento
- El objeto event

5. "Quiero…" → cómo se hace

6. Cadenas, números y arrays

7. Expresiones regulares

8. Boletín DOM resuelto
- DOM 1 · Tabla de filas × columnas numerada
- DOM 2 · Estilos según el select (change + setAttribute/removeAttribute)
- DOM 3 · Fondo negro / blanco
- DOM 4 · Tabla 3×3 de colores (delegación)

9. Boletín Eventos resuelto
- Ev 1 · Radio marcado → consola
- Ev 2 y 3 · Opción(es) elegida(s) de un select (simple o múltiple)
- Ev 4 · Dos listas múltiples (el HTML trae onclick="agregarProvincia()" etc.)
- Ev 5 · Añadir / quitar el manejador de un botón
- Ev 6 · Ratón sobre un cuadrado y teclas en un input
- Ev 7 · Botonera numérica con un único manejador
- Ev 8 · Impedir dígitos · Ev 9 · Impedir copiar
- Ev 10 · Qué botón del ratón
- Ev 11 · target, currentTarget y this
- Ev 12 · Cancelar el envío si el input está vacío
- Ev 13 · Validación solo con HTML

10. Ejercicios 14 y 15 explicados
- Ev 14 · Validar el formulario con JavaScript, con listas de vacíos y errores
- Ev 15 · Cookies: guardar, recuperar y eliminar

11. Ejercicio Alumnos (clases Alumno y Nota)
- Usar clases ya hechas
- Si el profe da una clase nueva: cómo leerla (primeros 5 minutos)
- Eventos y menú (codigo.js)
- Registrar alumno
- Rellenar los desplegables
- Registrar nota
- Detalle de un alumno (evento change)
- Notas medias
- Listado con botón Eliminar (review.js)

12. Ejercicio Final «TiendaDAW» resuelto (examen de prueba)
- Estructura del archivo y dónde va cada cosa
- Ap. 1 · Instanciar la clase desde datos normales
- Ap. 2 · Mostrar secciones y menú (delegación + data-seccion)
- Ap. 3a · Código automático (substring + filter + ceros)
- Ap. 3b · Alta con validaciones (submit, errores juntos, some, charAt)
- Ap. 4 y 6 · Pintar la tabla (template literals, filtros y orden)
- Ap. 5 · Botones de la tabla: delegación con data-accion y data-codigo
- ★ VENDER: find + método de la clase (true / false)
- ★ REPONER: método de la clase
- ★ ELIMINAR: confirm + findIndex + splice
- Ap. 6 · Tecla Escape y botón ordenar
- Ap. 7 · Resumen: sumas, el máximo y join
- Extra A · Herencia: extends, super e instanceof
- Fallos que tuve en este ejercicio (y su error en consola)

13. Errores típicos (revisar antes de entregar)

14. De las diapositivas, aún sin usar

15. Avance Unidad 5: fetch (no entra en el examen de DOM)
- Ejemplos con ejercicios hechos

★ Lo que pide el profesor (repaso rápido)

Conceptos que el profesor ha dicho que hay que tener claros, y dónde está cada uno en esta chuleta.

Concepto	Dónde
Eventos: click, change...	Apartado 4 (tipos de evento y objeto `event`) · ejercicios del 8 y 9
DOMContentLoaded	Aquí abajo ↓ (no salía en los ejercicios)
Delegación de eventos	Apartado 4 · DOM 4 · Ev 7 · Ev 11 · aquí abajo
Arrays: insertar, eliminar, buscar	Aquí abajo ↓ · Alumnos (push, splice, búsqueda con bucle)
Instanciar una clase / objetos	Aquí abajo ↓ · apartado 11 (Alumno y Nota, y cómo leer una clase que dé el profe)
Funciones de string	Aquí abajo ↓ (tabla completa) · apartado 6
Manejo de HTML	Apartados 2 y 3 · aquí abajo (template literals + innerHTML)
querySelector, etc.	Apartado 1
data-	Aquí abajo ↓ · Alumnos (botón Eliminar)

DOMContentLoaded

Salta cuando el navegador **ha terminado de leer todo el HTML** y ya existen todos los elementos (sin esperar a imágenes ni CSS). Sirve para que el script no se ejecute antes de tiempo.

```
<head>
  <script src="codigo.js"></script>    <!-- en el head: el body aún NO existe -->
</head>
```

```
// codigo.js
document.addEventListener("DOMContentLoaded", iniciar);   // en document, no en un elemento

function iniciar() {
    // aquí dentro el HTML ya existe: se pueden buscar elementos y poner eventos
    document.getElementById("boton").addEventListener("click", crearTabla);
    ocultarTodo();
}

// Versión con función anónima (también vale)
document.addEventListener("DOMContentLoaded", function () {
    document.getElementById("boton").addEventListener("click", crearTabla);
});
```

Dónde está el script	¿Hace falta DOMContentLoaded?
`<script>` en el `<head>`	**Sí.** Sin él, `getElementById` devuelve `null` → `TypeError: Cannot read properties of null (reading 'addEventListener')`
Al final del `<body>` (como en los ejercicios)	No: cuando se ejecuta, el HTML ya está leído. Ponerlo no hace daño
En el head con `defer`: `<script src="x.js" defer>`	No: `defer` espera a que el HTML esté leído

Cómo se escribe con `defer` (ejemplo con la clase del profesor + tu código):

```
<head>
  <!-- Con defer: los scripts se DESCARGAN mientras se lee el HTML,
       pero no se EJECUTAN hasta que todo el HTML está cargado.
       Por eso ya no haría falta DOMContentLoaded. -->

  <!-- 1º se ejecuta clases.js (define la clase Producto) -->
  <script src="js/clases.js" defer></script>

  <!-- 2º se ejecuta codigo.js (ya puede usar Producto y el DOM).
       Con defer se respeta el orden en que están escritos -->
  <script src="js/codigo.js" defer></script>
</head>
```

Ojo con defer: 1) solo funciona con scripts externos (`src="..."`); en `<script defer> código </script>` se ignora. 2) `clases.js` va **antes** que `codigo.js`; si no → `Producto is not defined`. 3) Es equivalente a poner los scripts al final del `body`. 4) Si el profe te da el HTML con los scripts en el head **sin** defer y no se puede tocar → `DOMContentLoaded`, que ponerlo nunca da error.

No confundir: `DOMContentLoaded` (HTML listo, se pone en `document`) con `load` (todo cargado, imágenes incluidas; se pone en `window`: `window.addEventListener("load", …)`).
Las **funciones** se pueden escribir fuera de `iniciar`; lo que va dentro es lo que **busca elementos** al arrancar.

Delegación de eventos (plantilla)

Cuándo usarla: cuando hay **muchos elementos que hacen lo mismo** (teclas de una calculadora, celdas de colores, botones Eliminar de cada fila) o cuando los elementos **se crean después** de cargar la página (filas añadidas con `innerHTML`). En vez de un `addEventListener` por elemento, uno en el contenedor. Ejemplos hechos: Calculadora, DOM 4, Ev 7.

```
// UN manejador en el contenedor, en vez de uno por botón/celda/fila
document.getElementById("contenedor").addEventListener("click", function (event) {
    if (event.target.tagName == "BUTTON") {                 // ¿se ha pulsado un botón?
        const usuario = event.target.dataset.usuario;        // ¿cuál? → data-usuario
        // ...
    }
});
// Otras formas de filtrar: event.target.classList.contains("btn-eliminar")
//                          event.target.matches("button.eliminar")
```

Funciona por la **propagación**: el clic sube del botón al contenedor. Ventajas: un solo `addEventListener`, y sirve para elementos creados **después** (filas nuevas de una tabla) sin volver a poner eventos.

Arrays: insertar, eliminar, buscar

Cuándo usar cada uno (Ejercicio Alumnos):

- `push` → al **registrar** un alumno.
- `some` → antes de registrar, para **no repetir usuario**.
- `find` → para **encontrar al alumno elegido** en el select (registrar nota, detalle) en vez de bucle + `if`.
- `findIndex` + `splice` → para **eliminar** (botón Eliminar).
- `filter` → para sacar un **grupo**: aprobados, alumnos con notas…
- `map` → para sacar **un dato de todos**: lista de nombres. `sort` → para **ordenar** (por media, por nombre).

Acción	Método	Ejemplo y qué devuelve

Insertar	`arr.push(x)`	Al **final** · `alumnos.push(alumno)`
	`arr.unshift(x)`	Al **principio**
	`arr.splice(i, 0, x)`	En la **posición i** (0 = no borra nada)
Eliminar	`arr.splice(i, 1)`	Quita **1** elemento en la posición `i` (cambia el array)
	`arr.pop()` · `arr.shift()`	Quita el **último** · el **primero** (y lo devuelve)
	`arr = arr.filter(cond)`	Crea un array **nuevo** sin los que no cumplen (la variable tiene que ser `let`)
	`arr.length = 0`	Vaciarlo entero
Buscar	`arr.find(cond)`	El **primer elemento** que cumple, o `undefined`
	`arr.findIndex(cond)`	Su **posición**, o `-1` (para luego hacer `splice`)
	`arr.filter(cond)`	**Todos** los que cumplen, en un array nuevo
	`arr.some(cond)`	`true` si **alguno** cumple (¿existe ya?)
	`arr.indexOf(x)` · `arr.includes(x)`	Solo para valores simples (números, textos): posición o `-1` · `true`/`false`
	Bucle `for` + `if`	Lo que hemos usado en los ejercicios: siempre funciona
Recorrer y otros	`for (const x of arr)` · `arr.forEach(x => …)`	Recorrer todos
	`arr.map(x => …)`	Array nuevo transformado: `alumnos.map(a => a.getNombre())`
	`arr.sort((a, b) => a - b)`	Ordenar números. Sin función ordena como texto: `[10,2,33].sort()` → `[10,2,33]`
	`arr.length` · `join(", ")` · `reverse()`	Tamaño · unir en texto · dar la vuelta

```
// La "condición" es una función flecha:  a => a.getUsuario() == usuario
// es lo mismo que:                      function (a) { return a.getUsuario() == usuario; }

// BUSCAR un alumno por usuario
const alumno = alumnos.find(a => a.getUsuario() == usuario);
if (alumno == undefined) { alert("No existe"); return; }

// ¿EXISTE ya? (evitar duplicados al registrar)
if (alumnos.some(a => a.getUsuario() == usuario)) { alert("Usuario repetido"); return; }

// ELIMINAR por usuario (dos formas)
const pos = alumnos.findIndex(a => a.getUsuario() == usuario);
if (pos != -1) alumnos.splice(pos, 1);
// o bien (con let alumnos):
alumnos = alumnos.filter(a => a.getUsuario() != usuario);

// FILTRAR: aprobados
const aprobados = notas.filter(n => n >= 5);
```

Lo que hay que hacer **después** de cambiar el array: volver a pintar lo que depende de él (tabla, desplegables).
`find` devuelve el **elemento**; `findIndex`, la **posición**; `filter`, un **array** (aunque solo haya uno).

Clases: crear e instanciar

Cuándo: crear una clase cuando el enunciado lo pide o hay que representar algo con **datos + comportamiento** (un Producto con su precio y un método que aplica descuento). **Instanciar** (`new`) cada vez que aparece uno nuevo: al registrar un alumno, al poner una nota, al cargar datos de un JSON. **Herencia** cuando hay un tipo especial de algo que ya existe (una Oferta es un Producto con descuento). **Privados + getters** cuando un dato no debe cambiarse desde fuera sin control (el `set` puede validar).

```
class Producto {
    #precio;                                    // atributo PRIVADO (solo se ve dentro de la
clase)

    constructor(nombre, precio) {               // se ejecuta al hacer new
        this.nombre = nombre;                   // atributo público
        this.#precio = precio;
    }

    getPrecio() { return this.#precio; }        // getter "estilo profe": se llama con ()
    setPrecio(valor) { if (valor >= 0) this.#precio = valor; }

    get precio() { return this.#precio; }       // getter con get: se usa SIN () → p.precio
    set precio(valor) { if (valor >= 0) this.#precio = valor; }   // p.precio = 100

    toString() { return this.nombre + " - " + this.#precio + " €"; }
}

// INSTANCIAR (crear un objeto de la clase)
const p = new Producto("Pala", 120);
p.getPrecio();      // 120
p.precio = 100;     // usa el set
p.nombre;           // "Pala" (público)
p.#precio;          // ERROR: privado
p instanceof Producto;   // true

// HERENCIA
class Oferta extends Producto {
    constructor(nombre, precio, descuento) {
        super(nombre, precio);                  // llama al constructor del padre (siempre lo
primero)
        this.descuento = descuento;
    }
    getPrecioFinal() { return this.getPrecio() * (1 - this.descuento / 100); }
}
const o = new Oferta("Raqueta", 80, 25);
o.getPrecioFinal();   // 60
```

Objeto literal	Objeto de una clase
`const a = { nombre: "Ana" }` · `a.nombre`	`const a = new Alumno("Ana", …)` · `a.getNombre()`
Rápido, sin métodos ni privados	Molde reutilizable: métodos, validación en el set, atributos privados

Errores típicos: olvidar `new` · el orden de los parámetros del constructor · getters de método sin `()` (`getNombre` devuelve la función) · leer un privado desde fuera (`undefined` o error) · en una clase hija, usar `this` antes de `super()`.

Funciones de string

> **Cuándo: construir datos** a partir de otros (usuario: `charAt`, `slice`, `split`, `toLowerCase`) · **validar** (`trim` + `== ""`, `length >= 6`, `includes("@")`, `startsWith`) · **separar** datos que vienen juntos (`"cliente:7.5".split(":")`, `"5*2".split("*")`) · **dar formato** (`toFixed(2)` en medias, `padStart` para `007`, template literals en mensajes y HTML).

Función	Ejemplo con `s = "Morales Pacheco"`
`s.length`	15
`s.toUpperCase()` · `toLowerCase()`	`"MORALES PACHECO"` · `"morales pacheco"`
`s.trim()`	Quita espacios del principio y del final
`s.charAt(0)` · `s[0]` · `s.at(-1)`	`"M"` · `"M"` · `"o"` (el último)
`s.slice(0, 3)` · `s.slice(-3)`	`"Mor"` · `"eco"` (desde el final)
`s.substring(0, 3)`	`"Mor"` (como slice, sin negativos)
`s.indexOf("P")` · `lastIndexOf`	8 · `-1` si no está
`s.includes("Pach")`	`true`
`s.startsWith("Mor")` · `endsWith("co")`	`true` · `true`
`s.split(" ")`	`["Morales", "Pacheco"]`
`s.replace("a", "4")` · `replaceAll("a", "4")`	`"Mor4les Pacheco"` (solo el primero) · `"Mor4les P4checo"`
`"7".padStart(3, "0")`	`"007"`
`"ab".repeat(3)`	`"ababab"`
`` `Hola ${nombre}` `` (comillas invertidas)	**Template literal**: mete variables dentro del texto sin `+`
`parseInt` · `parseFloat` · `Number("2.5")`	Texto → número
`(6.666).toFixed(2)` · `String(5)`	`"6.67"` (texto) · número → texto

> Los strings **no se modifican**: los métodos devuelven uno nuevo. `s.toUpperCase();` solo no cambia `s`; hay que guardarlo: `s = s.toUpperCase();`

Manejo de HTML con template literals

> **Cuándo usar cada forma:** `innerHTML` + template literal para **pintar rápido** listas y tablas a partir de un array (menos líneas); `createElement` + `appendChild` cuando necesitas **guardar el elemento** para ponerle eventos al crearlo, o cuando el texto viene **del usuario** (con `textContent` no se interpreta como HTML). Si pintas con `innerHTML` elementos que tienen que responder a clics → **delegación**.

```
// Crear filas con innerHTML (más corto que createElement, mismo resultado)
const tbody = document.getElementById("cuerpoTabla");
tbody.innerHTML = "";
for (const a of alumnos) {
    tbody.innerHTML += `<tr>
        <td>${a.getNombre()}</td>
        <td>${a.getCorreo()}</td>
        <td><button class="btn btn-danger" data-usuario="${a.getUsuario()}">Eliminar</button>
</td>
    </tr>`;
}
// Los botones creados así NO tienen eventos → usar DELEGACIÓN en tbody o en la tabla

elem.insertAdjacentHTML("beforeend", "<p>Hola</p>");   // añadir al final sin borrar lo
anterior
```

Atributos data-

> **Cuándo:** cuando un elemento tiene que **"recordar" a qué dato pertenece**: el botón Eliminar de cada fila (qué alumno), una celda (qué color o posición), una tarjeta de producto (qué id). Casi siempre va junto con la **delegación**: un solo manejador y `event.target.dataset.x` para saber cuál se ha pulsado.

```
<button data-usuario="rmorpac" data-id-nota="3">Eliminar</button>
```

En JavaScript	Resultado
`boton.dataset.usuario`	`"rmorpac"` (sin el `data-`)
`boton.dataset.idNota`	`"3"` — con guiones pasa a camelCase (`data-id-nota` → `idNota`). Siempre es **texto**: `parseInt` si es número
`boton.dataset.usuario = "agarper"`	Lo crea o lo cambia (en el HTML: `data-usuario="agarper"`)
`boton.getAttribute("data-usuario")` · `setAttribute("data-usuario", …)`	Lo mismo con los métodos de atributos
`document.querySelector('[data-usuario="rmorpac"]')`	Buscar un elemento por su data-

Para qué: guardar en un elemento un dato propio (usuario, id, posición…) para saber, al pulsarlo, a qué objeto se refiere. Es la pareja perfecta de la delegación: `event.target.dataset.usuario`.

0. Método para empezar cualquier ejercicio

#	Pregunta	Se traduce en…
1	**¿Cuándo** pasa algo?	Un `addEventListener` sobre el elemento que **se pulsa / dispara**
2	**¿Qué datos** necesito?	Leer valores (`.value`, `.checked`, `event.key`…) **dentro** de la función
3	**¿Sobre qué elemento** actúo?	El elemento que **cambia**: seleccionarlo y guardarlo en una variable
4	**¿Qué le hago?**	Las acciones, con `if`/`switch` si dependen de algo

Disparador ≠ objetivo. Lo que se pulsa lleva el evento; lo que cambia es otra cosa. Ej.: se pulsa el botón "Marcar/Desmarcar", cambia el checkbox. Se pulsa una celda, cambia el `body`.
Antes de escribir: esqueleto con comentarios (`// 1. leer… // 2. crear…`) y luego cada paso a una línea.

1. Seleccionar elementos

Método	Devuelve	Ojo
`getElementById("id")`	**Un** elemento o `null`	Sin `#`
`querySelector("selector")`	El **primero** o `null`	`#id`, `.clase`, `input[name=x]`, `form[name=formulario]`
`querySelectorAll("selector")`	**Lista estática** (una foto)	Recorrer con `for...of`
`getElementsByName("valor")`	**Lista viva**	Solo existe en `document`. Entre paréntesis el **valor** del name
`getElementsByClassName("clase")`	**Lista viva**	Sin punto. Uno solo: `[0]`
`getElementsByTagName("p")`	**Lista viva**	La etiqueta: `"input"`, `"td"`…
`document.body` · `document`	El body · la página entera	No hace falta buscarlos

`Elements` **en plural o** `All` **→ lista** → no tiene `addEventListener`, `.value` ni `appendChild`.
Selectores CSS útiles: espacio = "dentro de" (`"#lstProvincias option"`, `"#capa td"`) · coma = "o" (`"#a option, #b option"`) · `etiqueta[atributo=valor]`.
Mayúsculas: el valor del atributo distingue mayúsculas: `name="Apellidos"` ≠ `input[name=apellidos]`.
No usar atajos tipo `formulario.algo` (el name como variable): usar siempre la ruta completa.

Colecciones vivas vs estáticas

Viva (cambia mientras la recorres)	Estática (foto)
`getElementsBy*`, recorrer un `<select>`, `select.options`, `elem.children`	`querySelectorAll`

Si vas a **mover o borrar** elementos dentro de un bucle, recorre una lista **estática**: con una viva, al sacar uno los demás se desplazan y **te saltas alguno**.

2. Leer y modificar elementos

Quiero…	Cómo
Lo escrito en `input`/`select`/`textarea`	`elem.value` (siempre **texto**). En un select, el value de la opción elegida
El texto de `p`, `td`, `button`, `option`	`elem.textContent` (en option también `.text`)
HTML de dentro / vaciar / saltos de línea	`elem.innerHTML` · `innerHTML = ""` · ` ` · `innerHTML +=` para acumular

¿Radio o checkbox marcado?	`elem.checked` (se lee y se cambia). El `value` es fijo
¿Opción de un select elegida?	`option.selected` · `select.selectedIndex` · `select.options[i]`
Atributos	`setAttribute("style", "color: blue; font-size: 20px")` · `getAttribute("href")` · `removeAttribute("class")` · `hasAttribute("x")`
Un estilo	`elem.style.backgroundColor = "black"` (camelCase: `fontSize`, `fontFamily`)
Clases	`classList.add("x")` · `remove("x")` · `toggle("x")` · `contains("x")` · `className = "x"`
CSS externo	`document.getElementById("estilo").setAttribute("href", "ej08.css")`
Ocultar / mostrar / desactivar	`elem.hidden = true/false` · `elem.disabled = true`
Vaciar un formulario	`formulario.reset()`

value vs textContent: un input tiene **valor**; un `p`/`td`/`div` tiene **texto**. Un `<form>` no tiene value.
Elemento / atributo / valor: `elemento.setAttribute("atributo", "valor")`. `<link>` es el elemento, `href` el atributo.
Valor de style: un único texto CSS: `"color: blue; font-size: 20px; font-family: Helvetica"` (`20px` junto).
Clases sin punto: `classList.add("amarillo")`, `setAttribute("class", "claseEstilo")`. El punto solo en CSS y selectores.
Prioridad: el `style` del propio elemento gana a una clase.
Fuente: `font-size`, `font-family`, `font-weight`, `font-style`. `width`/`height` son la caja.

3. Crear, mover y borrar elementos

Método	Para qué
`document.createElement("td")`	Crea en **memoria**. Nombre sin `< >`. Siempre con `document.`
`padre.appendChild(hijo)`	Mete al final. Solo **elementos**. Si el hijo ya está en la página, lo **mueve**
`padre.append(a, "texto")` · `prepend` · `before` · `after`	Variantes (admiten texto)
`tabla.insertRow()` · `fila.insertCell()`	Atajos de tablas
`elem.remove()` · `elem.cloneNode(true)`	Quitar · copiar (true = con hijos)

Regla: si el elemento **ya existe** en el HTML → se coge y se modifica. Si **lo creas tú** → `createElement` + `appendChild`.
No encadenar: `createElement("tr").appendChild(celda)` devuelve la celda. Crear y meter en dos líneas.
Contenedor: un `div` vacío con id en el HTML es el hueco donde va lo que genera tu JS.
Trampa VS Code: escribir `createElement` sin `document.` mete `require("react")` arriba y rompe todo.

```
// Crear una opción y meterla en un select:  <option value="SE">Sevilla</option>
const opt = document.createElement("option");
opt.value = codigo;           // lo que se envía
opt.textContent = provincia;  // lo que se ve
select.appendChild(opt);
```

4. Eventos

Registrar y quitar

Forma	Notas
`elem.addEventListener("click", miFuncion)`	**No devuelve nada**: nunca `const x = … addEventListener(…)`. `miFuncion` sin paréntesis
`elem.removeEventListener("click", miFuncion)`	Mismo tipo y **misma función con nombre**
`onclick="miFuncion()"` en el HTML	Hace lo mismo: solo hay que definir la función. No usarlo a la vez que addEventListener

> **miFuncion** = pasarla para **más tarde** · **miFuncion()** = ejecutarla **ahora**.
> El `addEventListener` puede ir **dentro de otra función**: el evento se pone cuando esa función se ejecuta.
> Leer los datos **dentro** de la función del evento, nunca al cargar (los inputs estarían vacíos).

Tipos de evento

Evento	Salta cuando…	Dónde se pone
`click`	Clic con el botón izquierdo	Botón, celda, contenedor (delegación)
`change`	Cambia la opción de un select / el valor de un input	El `<select>`
`submit`	Se va a enviar el formulario (botón submit o Enter)	El **`<form>`**, no el botón
`mouseover` / `mouseout`	El ratón entra / sale (también `mouseenter`/`mouseleave`)	El elemento
`mousedown`	Se pulsa **cualquier** botón del ratón	`document` para toda la página
`contextmenu`	Clic derecho (abre el menú)	`document`
`keydown`	Se pulsa una tecla	El input donde se escribe
`copy`	Se copia (Ctrl+C o clic derecho)	El input

El objeto event

Propiedad / método	Qué es
`event.target`	Dónde se hizo clic de verdad (cambia según dónde pulses)
`event.currentTarget` · `this`	El elemento que **tiene** el manejador (fijo). `this` va solo, no `event.this`
`event.target.tagName`	Etiqueta en MAYÚSCULAS: `"BUTTON"`, `"INPUT"`, `"TD"`, `"P"` (N mayúscula)
`event.type`	Tipo: `"mouseover"`, `"click"`…
`event.clientX` / `clientY`	Coordenadas del ratón en la ventana
`event.key`	Tecla pulsada: `"a"`, `"5"`, `"Enter"`, `"Shift"`

`event.button`	0 izquierdo · 1 rueda · 2 derecho
`event.preventDefault()`	Cancela la acción por defecto (escribir la tecla, copiar, abrir menú, enviar el form)
`event.stopPropagation()`	El evento deja de subir a los padres

> **Truco:** `console.log(event)` dentro de la función y desplegarlo en la Consola para ver todas sus propiedades.
> **Propagación:** el evento sube del hijo a los padres (P → DIV → FORM → BODY → document). Por eso un manejador en un contenedor se entera de los clics de dentro.
> **Delegación:** un solo `addEventListener` en el contenedor + `if (event.target.tagName == "…")`. Sirve también para elementos que **aún no existen** al cargar (creados con JS).

5. "Quiero…" → cómo se hace

Quiero…	Receta	Ejercicio
Generar una tabla	Bucles anidados: `tr` en el de filas, `td` en el de columnas; contador; vaciar contenedor y meter la tabla al final	DOM 1, 4
Aplicar estilos según un select	`change` + `select.value` + `switch`; función `limpiar()` al principio	DOM 2
Alternar entre dos estados	`if/else` mirando el estado actual, o `classList.toggle` + `contains`. Nunca dos `if` seguidos	DOM 3
Saber qué radio está marcado	`for (const r of getElementsByName("x")) if (r.checked) …`	Ev 1
Saber qué opciones de un select están elegidas	`for (const o of select) if (o.selected) …`	Ev 2, 3
Mover opciones entre dos selects	`querySelectorAll("#origen option")` + `if (o.selected) destino.appendChild(o)`	Ev 4
Activar / desactivar un botón	`addEventListener` / `removeEventListener` dentro de funciones	Ev 5
Reaccionar al ratón / teclado	`mouseover`/`mouseout`, `keydown` + propiedades de event	Ev 6
Teclado o botonera con muchos botones	Delegación en el div + `event.target.value` + `pantalla.value +=`	Ev 7
Impedir escribir ciertas teclas	`keydown` + `if ("0123456789".includes(event.key)) event.preventDefault()`	Ev 8
Impedir copiar / menú contextual	`copy` / `contextmenu` + `preventDefault()`	Ev 9, 10
Que no se envíe un formulario inválido	`submit` en el form + `preventDefault()` si algo falla	Ev 12, 14
Validar formato	HTML: `required pattern="…"` · JS: `/^…$/.test(valor)`	Ev 13, 14
Guardar datos en el navegador	`document.cookie` (ver ejercicio 15)	Ev 15
Usar clases ya hechas	`new Clase(...)` + getters con `()`	Alumnos

Mostrar solo una sección del menú	`ocultarTodo()` + `style.display = "block"` según `event.target.id`	Alumnos
Botón Eliminar en cada fila	`dataset` + `confirm` + `splice` + volver a pintar	Alumnos
Validar antes de guardar	Comprobaciones con `alert` + `return` al principio; lo bueno al final	Ev 4, formulario

6. Cadenas, números y arrays

Elemento	Ejemplo
`cad.length`	Nº de caracteres: `"3".length` → 1 (¡no es el número 3!)
`cad.charAt(0)` · `cad[0]`	Primer carácter (charAt con paréntesis)
`cad.slice(0, 3)`	`"Morales".slice(0,3)` → `"Mor"`
`cad.split(" ")`	`"Morales Pacheco".split(" ")[1]` → `"Pacheco"` · `"5*2".split("*")[1]` → `"2"`
`grande.includes(pequeña)`	`"0123456789".includes("5")` → true. En JS no existe `contains`
`trim()` · `toUpperCase()` · `toLowerCase()`	Quitar espacios de los lados · mayúsculas · minúsculas
`parseInt("12")` · `parseFloat("2.5")`	Texto → número (parseInt corta decimales)
`arr.push(x)` · `arr.join(" ")` · `arr.length`	Añadir · unir en texto · tamaño
`find` · `filter` · `findIndex` · `splice(i, 1)`	Buscar · filtrar · posición · borrar

`"5" + "55"` → `"555"` (texto + texto pega; convertir antes de sumar). `"5" - "-"` → `NaN`.
Un solo `=` asigna (el if siempre entra); comparar con `===`/`==`.
`isNaN(" ")` es false: el espacio "pasa" como número.

```
// Objeto y array de objetos
let alumnos = [];                          // FUERA de las funciones (si no, se vacía en cada clic)
const alumno = { nombre: "Ana", correo: "ana@x.com" };
alumnos.push(alumno);
alumno.nombre;  alumno["correo"];
```

7. Expresiones regulares

Pieza	Significa	Pieza	Significa
`[0-9]`	Un dígito	`+`	Una o más veces
`[a-z]` `[A-Z]`	Minúscula / mayúscula	`*`	Cero o más veces
`[6-9]`	Un carácter del 6 al 9	`( )`	Grupo
`{3}` `{2,4}`	Exactamente 3 / entre 2 y 4	`|`	"o"
`\.`	Punto literal	`.`	Cualquier carácter

Campo	Expresión (atributo `pattern`)	Ejemplo válido

Nombre / apellidos	`[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+( [A-ZÁÉÍÓÚÑ][a-záéíóúñ]+)*`	Pérez López
Fecha DD/MM/AAAA	`(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/[0-9]{4}`	17/01/2023
DNI	`[0-9]{7,8}[A-Z]`	12345678Z
Correo	`[a-z._\-]+@[a-z_\-]+\.[a-z]{2,4}`	correo.prueba@dominio.info
Usuario IDEA	`[a-z]{7}[0-9]{3}`	rmorpac123
Teléfono	`[6-9][0-9]{8}`	612345678
Twitter	`@.{4,15}`	@moha_daw
Código postal	`[0-9]{5}`	41001

HTML (ej. 13)	**JavaScript (ej. 14)**
`<input required pattern="[0-9]{7,8}[A-Z]">`	`/^[0-9]{7,8}[A-Z]$/.test(valor)`
Comprueba el texto entero solo	Hay que poner `^` y `$`
Guion dentro de `[ ]` escapado: `\-` (si no, Chrome ignora la expresión)	La `/` literal se escapa: `\/`
Valida el navegador al enviar	Validas tú en el `submit`

Equivalencias con Spring: `@NotBlank` → `required` · `@Pattern` → `pattern` · `@Size` → `minlength`/`maxlength`.

8. Boletín DOM resuelto

DOM 1 · Tabla de filas × columnas numerada

```
document.getElementById("boton").addEventListener("click", crearTabla);

function crearTabla() {
    const filas = parseInt(document.querySelector("input[name=filas]").value);
    const columnas = parseInt(document.querySelector("input[name=columnas]").value);
    const tabla = document.createElement("table");
    let numero = 0;
    for (let i = 0; i < filas; i++) {
        const fila = document.createElement("tr");
        for (let j = 0; j < columnas; j++) {
            const celda = document.createElement("td");
            numero++;
            celda.textContent = numero;
            fila.appendChild(celda);
        }
        tabla.appendChild(fila);                // la fila, cuando está completa
    }
    const contenedor = document.getElementById("tabla");
    contenedor.innerHTML = "";                  // no se acumulan tablas
    contenedor.appendChild(tabla);              // una vez, al final
}
```

DOM 2 · Estilos según el select (change + setAttribute/removeAttribute)

```
document.querySelector("select[name=opciones]").addEventListener("change", opcionDesplegable);
const texto = document.getElementById("texto");    // elementos: pueden ir fuera
const enlace = document.getElementById("estilo");  // el <link>

function limpiar() {
    texto.removeAttribute("style");
    texto.removeAttribute("class");
    enlace.removeAttribute("href");
}

function opcionDesplegable() {
    const opcion = document.querySelector("select[name=opciones]").value;  // valor: dentro
    limpiar();
    switch (opcion) {
        case "atributoStyle":   texto.setAttribute("style", "color: blue; font-size: 20px;
font-family: Helvetica"); break;
        case "asignandoClases": texto.setAttribute("class", "claseEstilo"); break;
        case "estilosPagina":   enlace.setAttribute("href", "ej08.css"); break;
        // "quitarEstilos": ya lo hace limpiar()
    }
}
```

> Variables con `const`/`let` solo existen en su función: para usarlas en otra, pasarlas como parámetro o declararlas fuera. Fuera se pueden declarar **elementos**; los **valores** se leen dentro. Un parámetro tapa a una variable de fuera con el mismo nombre.

DOM 3 · Fondo negro / blanco

```
// Versión con style (estado inicial fijado al cargar)
document.getElementById("boton").addEventListener("click", fondos);
document.body.style.background = "white";

function fondos() {
    const boton = document.getElementById("boton");
    if (document.body.style.background == "white") {
        document.body.style.background = "black";
        boton.textContent = "Fondo Blanco";
    } else {                                        // else, NO dos if seguidos
        document.body.style.background = "white";
        boton.textContent = "Fondo Negro";
    }
}

// Versión con clase (.oscuro { background-color: black; } en el CSS)
function fondos() {
    const boton = document.getElementById("boton");
    document.body.classList.toggle("oscuro");
    boton.textContent = document.body.classList.contains("oscuro") ? "Fondo Blanco" : "Fondo
Negro";
}
```

DOM 4 · Tabla 3×3 de colores (delegación)

```
document.getElementById("boton").addEventListener("click", colorCelda);
document.getElementById("capa").addEventListener("click", cambiarFondo);   // capa ya existe

function colorCelda() {
    const colores = ["#ff0000", "#00ff00", "#0000ff", "#ffff00", "#ff00ff",
                     "#00ffff", "#ffa500", "#808080", "#000000"];
    let contador = 0;
    const capa = document.getElementById("capa");
    const tabla = document.createElement("table");
    capa.innerHTML = "";
    for (let i = 0; i < 3; i++) {
        const fila = document.createElement("tr");
        tabla.appendChild(fila);
        for (let j = 0; j < 3; j++) {
            const celda = document.createElement("td");
            celda.style.backgroundColor = colores[contador];
            contador++;
            fila.appendChild(celda);
        }
    }
    capa.appendChild(tabla);
}

function cambiarFondo(event) {
    if (event.target.tagName == "TD") {
        document.body.style.backgroundColor = event.target.style.backgroundColor;
    }
}
```

En el CSS: `td { width: 50px; height: 50px; }` (las reglas se aplican también a lo creado con JS).

9. Boletín Eventos resuelto

Ev 1 · Radio marcado → consola

```
document.getElementById("consultar").addEventListener("click", consultarDatos);
function consultarDatos() {
    for (const input of document.getElementsByName("actores")) {
        if (input.checked) console.log(input.value);
    }
}
```

Ev 2 y 3 · Opción(es) elegida(s) de un select (simple o múltiple)

```
document.querySelector("input[name=boton]").addEventListener("click", mostrarInfo);
function mostrarInfo() {
    const select = document.querySelector("select[name=provincias]");
    for (const provincia of select) {
        if (provincia.selected) console.log(provincia.textContent + " - " + provincia.value);
    }
}
// Sin bucle (solo la primera): select.options[select.selectedIndex]
```

Ev 4 · Dos listas múltiples (el HTML trae onclick="agregarProvincia()" etc.)

```
function agregarProvincia() {
    const codigo = document.getElementById("txtCodigo").value.trim();
    const provincia = document.getElementById("txtProvincia").value.trim();
    if (codigo === "" || provincia === "") { alert("Rellena los dos campos"); return; }
    for (const o of document.querySelectorAll("#lstProvincias option, #lstDestino option")) {
        if (o.value === codigo) { alert("La provincia ya existe"); return; }
    }
    const opt = document.createElement("option");
    opt.value = codigo;
    opt.textContent = provincia;
    document.getElementById("lstProvincias").appendChild(opt);
}

function pasarDerecha() {
    const destino = document.getElementById("lstDestino");
    for (const o of document.querySelectorAll("#lstProvincias option")) {   // foto estática
        if (o.selected) destino.appendChild(o);                              // appendChild MUEVE
    }
}

function pasarIzquierda() {
    const lista = document.getElementById("lstProvincias");
    for (const o of document.querySelectorAll("#lstDestino option")) {
        if (o.selected) lista.appendChild(o);
    }
}
```

Ev 5 · Añadir / quitar el manejador de un botón

```
const marcar = document.getElementById("verano");          // el checkbox (lo que cambia)
function marcarVerano() { marcar.checked = !marcar.checked; }
function addManejador() {
    document.getElementById("botonMarcar").addEventListener("click", marcarVerano);
}
function deleteManejador() {
    document.getElementById("botonMarcar").removeEventListener("click", marcarVerano);
}
```

Ev 6 · Ratón sobre un cuadrado y teclas en un input

```
const cuadrado = document.getElementById("cuadrado");
cuadrado.addEventListener("mouseover", pintar);
cuadrado.addEventListener("mouseout", despintar);
document.getElementById("txtEntrada").addEventListener("keydown", tecla);

function pintar(event) {
    cuadrado.classList.add("amarillo");
    document.getElementById("salida").textContent = "Evento: " + event.type + " - Objeto: " +
        event.target.id + " - X: " + event.clientX + " - Y: " + event.clientY;
}
function despintar() {
    cuadrado.classList.remove("amarillo");
    document.getElementById("salida").textContent = "";
}
function tecla(event) {
    document.getElementById("salida").innerHTML += "Tecla pulsada: " + event.key + "<br>";
}
```

Ev 7 · Botonera numérica con un único manejador

```
document.getElementById("teclado").addEventListener("click", tecla);
function tecla(event) {
    if (event.target.tagName == "INPUT") {            // <input type="button"> → "INPUT"
        document.getElementById("salida").value += event.target.value;
    }
}
```

Ev 8 · Impedir dígitos · Ev 9 · Impedir copiar

```
document.getElementById("txtEntrada").addEventListener("keydown", function (event) {
    if ("0123456789".includes(event.key)) event.preventDefault();
});
document.getElementById("txtEntrada").addEventListener("copy", function (event) {
    event.preventDefault();
});
```

Ev 10 · Qué botón del ratón

```
document.addEventListener("mousedown", mostrarBoton);
document.addEventListener("contextmenu", function (event) { event.preventDefault(); });
function mostrarBoton(event) {
    const salida = document.getElementById("salida");
    if (event.button == 0) salida.textContent = "Has pulsado el botón izquierdo";
    else if (event.button == 2) salida.textContent = "Has pulsado el botón derecho";
}
```

Ev 11 · target, currentTarget y this

```
document.getElementById("formulario").addEventListener("click", function (event) {
    console.log(event.target.tagName);         // P, DIV o FORM: donde se hizo clic
    console.log(event.currentTarget.tagName);  // FORM siempre: quien tiene el manejador
    console.log(this);                         // <form> siempre
});
```

Ev 12 · Cancelar el envío si el input está vacío

```
document.querySelector("form[name=formulario]").addEventListener("submit", enviarForm);
function enviarForm(event) {
    const texto = document.querySelector("input[name=txtTexto]").value.trim();
    if (texto === "") {
        alert("No hay texto, no se puede enviar");
        event.preventDefault();
    }
}
```

Ev 13 · Validación solo con HTML

Añadir a cada input `required` y `pattern="…"` con las expresiones del apartado 7. Ejemplo: `<input type="text" name="telefono" required pattern="[6-9][0-9]{8}" />`. Sin JavaScript.

10. Ejercicios 14 y 15 explicados

Ev 14 · Validar el formulario con JavaScript, con listas de vacíos y errores

Enunciado: validar el formulario del ejercicio 13 desde JS. En la capa de salida se muestran una lista con los campos **vacíos** y otra con los campos **con errores**. Si todo es correcto, el formulario se envía.

```
document.querySelector("form[name=formulario]").addEventListener("submit", validarFormulario);

// Un objeto por campo: su name, el texto que se muestra y la expresión que debe cumplir
const campos = [
    { nombre: "apellidos",       texto: "Apellidos",           regex: /^[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+
( [A-ZÁÉÍÓÚÑ][a-záéíóúñ]+)*$/ },
    { nombre: "nombre",          texto: "Nombre",              regex: /^[A-ZÁÉÍÓÚÑ][a-záéíóúñ]+
( [A-ZÁÉÍÓÚÑ][a-záéíóúñ]+)*$/ },
    { nombre: "fechaNacimiento", texto: "Fecha de nacimiento", regex: /^(0[1-9]|[12][0-
9]|3[01])\/(0[1-9]|1[0-2])\/[0-9]{4}$/ },
    { nombre: "dni",             texto: "DNI",                 regex: /^[0-9]{7,8}[A-Z]$/ },
    { nombre: "email",           texto: "Correo electrónico",  regex: /^[a-z._-]+@[a-z_-]+\.[a-
z]{2,4}$/ },
    { nombre: "usuario",         texto: "Usuario IDEA",        regex: /^[a-z]{7}[0-9]{3}$/ },
    { nombre: "telefono",        texto: "Teléfono",            regex: /^[6-9][0-9]{8}$/ },
    { nombre: "twitter",         texto: "Usuario de Twitter",  regex: /^@.{4,15}$/ }
];

function validarFormulario(event) {
    const vacios = [];
    const errores = [];

    for (const campo of campos) {
        const valor = document.querySelector("input[name=" + campo.nombre + "]").value.trim();
        if (valor === "") {
            vacios.push(campo.texto);
        } else if (!campo.regex.test(valor)) {
            errores.push(campo.texto);
        }
    }

    const salida = document.getElementById("salida");
    salida.innerHTML = "";

    if (vacios.length === 0 && errores.length === 0) {
        return;                                  // todo bien: no se cancela y se envía
    }

    event.preventDefault();                      // algo falla: no se envía
    if (vacios.length > 0)  salida.appendChild(crearLista("Campos vacíos:", vacios));
    if (errores.length > 0) salida.appendChild(crearLista("Campos con errores:", errores));
}

// Devuelve un div con un título y una lista <ul> con un <li> por elemento
function crearLista(titulo, elementos) {
    const bloque = document.createElement("div");
    const p = document.createElement("p");
    p.textContent = titulo;
    bloque.appendChild(p);
    const ul = document.createElement("ul");
    for (const elemento of elementos) {
        const li = document.createElement("li");
        li.textContent = elemento;
        ul.appendChild(li);
    }
    bloque.appendChild(ul);
    return bloque;
}
```

Cómo funciona:

1. **Array de objetos `campos`:** en vez de 8 bloques `if` casi iguales, cada campo se describe con su `name`, su texto y su expresión. Un solo bucle los valida todos; para añadir un campo basta con una línea más.

2. **Evento `submit` en el formulario** (como el ej. 12): salta con el botón y con Enter, antes de enviar.
3. **El bucle:** construye el selector con el name (`"input[name=" + campo.nombre + "]"`), lee el valor con `trim()` y lo clasifica: si está vacío va a `vacios`; si no está vacío pero `regex.test(valor)` da `false`, va a `errores`. El `else if` evita que un campo vacío cuente también como error.
4. **Decisión:** si los dos arrays están vacíos, `return` y el navegador envía el formulario. Si no, `preventDefault()` y se pintan las listas.
5. **`crearLista`:** función auxiliar reutilizable; mismo patrón que la tabla (crear, rellenar, `appendChild`), pero con `ul`/`li`. Se vacía la salida al principio para no acumular mensajes de intentos anteriores.
6. **Expresiones en JS:** entre `/ /`, con `^` (principio) y `$` (final) para que se compruebe el texto entero, y la `/` de la fecha escapada como `\/`.

Ev 15 · Cookies: guardar, recuperar y eliminar

Enunciado: tres botones. **Guardar** crea la cookie clave=valor con lo escrito; **Recuperar** muestra en un `alert` el valor de la cookie cuya clave se ha escrito; **Eliminar** borra la cookie de esa clave. Después de cada acción se resetea el formulario.

```
document.querySelector("input[name=btnGuardar]").addEventListener("click", guardar);
document.querySelector("input[name=btnRecuperar]").addEventListener("click", recuperar);
document.querySelector("input[name=btnEliminar]").addEventListener("click", eliminar);
const formulario = document.querySelector("form[name=formulario]");

function guardar() {
    const clave = document.querySelector("input[name=txtClave]").value.trim();
    const valor = document.querySelector("input[name=txtValor]").value.trim();
    if (clave === "") { alert("Debes escribir una clave"); return; }
    document.cookie = encodeURIComponent(clave) + "=" + encodeURIComponent(valor) +
                      "; max-age=" + 60 * 60 * 24 * 7 + "; path=/";    // 7 días
    formulario.reset();
}

function recuperar() {
    const clave = document.querySelector("input[name=txtClave]").value.trim();
    if (clave === "") { alert("Debes escribir una clave"); return; }
    const valor = leerCookie(clave);
    if (valor === null) alert("No existe ninguna cookie con la clave " + clave);
    else alert("El valor de la cookie " + clave + " es: " + valor);
    formulario.reset();
}

function eliminar() {
    const clave = document.querySelector("input[name=txtClave]").value.trim();
    if (clave === "") { alert("Debes escribir una clave"); return; }
    if (leerCookie(clave) === null) {
        alert("No existe ninguna cookie con la clave " + clave);
    } else {
        document.cookie = encodeURIComponent(clave) + "=; max-age=0; path=/";   // caduca ya
        alert("Cookie " + clave + " eliminada");
    }
    formulario.reset();
}

// document.cookie = "clave1=valor1; clave2=valor2" → partir y buscar
function leerCookie(clave) {
    for (const cookie of document.cookie.split("; ")) {
        const partes = cookie.split("=");
        if (decodeURIComponent(partes[0]) === clave) return decodeURIComponent(partes[1]);
    }
    return null;
}
```

Cómo funciona `document.cookie`:

Acción	Explicación
Guardar	`document.cookie = "clave=valor; max-age=SEGUNDOS; path=/"`. Aunque parezca que machaca, cada asignación **añade o sobrescribe una sola** cookie y deja las demás. `max-age` en segundos; sin él, la cookie desaparece al cerrar el navegador. `path=/` la hace visible en todo el sitio.
Leer	Al leer `document.cookie` se obtienen **todas** juntas en un texto: `"color=azul; idioma=es"`. Hay que partir por `"; "` y cada trozo por `"="`. Por eso existe `leerCookie`, que devuelve el valor o `null`.
Eliminar	No hay método para borrar: se guarda la **misma clave** con `max-age=0` (o `expires` en una fecha pasada) y caduca al momento.
encode / decode	`encodeURIComponent` convierte espacios, `;` y `=` para que no rompan el formato; `decodeURIComponent` lo deshace al leer.
reset()	Método del formulario que vacía todos sus campos de golpe.

⚠️ **Probar con Live Server** (`http://127.0.0.1…`): abriendo el HTML con doble clic (`file://`) Chrome no guarda cookies y parece que no funciona.

11. Ejercicio Alumnos (clases Alumno y Nota)

Qué es: una aplicación con menú (Registro, Registrar Notas, Listado, Notas Medias, Cerrar Sesión) que usa las clases `Alumno` y `Nota` que da el profe en `clases.js`. El código va en `codigo.js` y el listado en `review.js`.

Usar clases ya hechas

Qué	Cómo
Crear un objeto	`new Alumno(nombre, apellidos, usuario, contrasena, correo)` · `new Nota(asignatura, nota)` — en el orden del constructor
Leer datos privados (`#nombre`)	Solo con getters **y con paréntesis**: `alumno.getNombre()`. `alumno.nombre` da `undefined`
Métodos de `Alumno`	`getNombre()` · `getApellidos()` · `getUsuario()` · `getContrasena()` · `getCorreo()` · `getLongitudArrayNotas()` · `getNota(i)` · `getAsignaturaYNota(i)` → `"cliente:7.5"` · `altaNota(oNota)` → `true` si **ya existía** (y no la guarda)
Métodos de `Nota`	`getAsignatura()` · `getNota()` · `setAsignatura(v)` · `setNota(v)`
Ver un objeto en consola	`console.log(alumno)` sale como `Alumno {}` (los privados no se ven). Comprobar con `alumnos[0].getNombre()`

Un objeto `Alumno` no es un elemento HTML: no se puede meter con `appendChild`; hay que crear `<option>`, `<td>`… con sus datos.
Una misma `<option>` no puede estar en dos selects: crear una para cada uno.
`reset()` solo existe en `<form>`, no en selects ni botones.
Ids parecidos: `selAlumnoDetalle` es el select; `detalleAlumno` es el div de la tabla.
Varios scripts: se cargan en orden (`clases.js` → `codigo.js` → `review.js`) y comparten lo declarado fuera de las funciones.

VS Code no sugiere los métodos de `alumno.` ni de `event.` porque no sabe su tipo (el array empieza vacío, la clase está en otro archivo). El icono `abc` son solo palabras del archivo. **El código**

funciona igual escribiéndolo a mano. Para que sugiera: `/** @type {Alumno[]} */` encima de `let alumnos = [];` · `/** @param {Event} event */` (o `MouseEvent`, `KeyboardEvent`) encima de la función · `/** @param {string} cadena */` para un texto.

Si el profe da una clase nueva: cómo leerla (primeros 5 minutos)

Lo más probable en el examen es que dé la clase hecha y haya que **usarla**. Antes de programar, leerla con estas preguntas (apuntarlas como comentario al principio del JS):

#	Pregunta	Dónde mirar	Ejemplo en `clases.js`
1	¿Qué datos pide para crear un objeto, y en qué **orden**?	El `constructor(...)`	`Alumno(n, a, u, c, co)` · `Nota(a, n)`
2	¿Qué atributos son **privados**?	Los que empiezan por `#`	Todo `Alumno` → solo getters
3	¿Qué métodos tiene y cuáles **devuelven** algo?	Los `return`	`getNombre()` devuelve texto; `altaNota()`, true/false
4	¿Algún método **recibe otro objeto**?	Los parámetros	`altaNota(oNota)` recibe una `Nota` → antes `new Nota(...)`
5	¿Qué **significa** lo que devuelve?	Comentarios y código del método	`altaNota`: `true` = ya existía y **no** la guarda

Reglas al usarla: no modificar la clase (tu código va en tu archivo) · guardar en el array **objetos de la clase**, no objetos `{ }` · **guardar lo que devuelven** los métodos en una variable y decidir con `if` (`const existe = alumno.altaNota(nota)`). El resto (orden del constructor, getters con `()`, no es HTML) está en la tabla y el recuadro de arriba.

```
// Probarla en la CONSOLA del navegador (F12) antes de escribir tu código:
const a = new Alumno("Ana", "Gil Ruiz", "agilrui", "Abc123", "a@x.com");
a.getNombre();                          // "Ana"
a.altaNota(new Nota("cliente", 7));     // false → la ha guardado
a.altaNota(new Nota("cliente", 9));     // true  → ya existía
a.getLongitudArrayNotas();              // 1
```

Eventos y menú (codigo.js)

```
document.getElementById("btnCrearUsuario").addEventListener("click", crearUsuario);
document.getElementById("btnRegistrarAlumno").addEventListener("click", registrarAlumno);
document.getElementById("registroAlumnos").addEventListener("click", mostrarRegistro);
document.getElementById("registroNotas").addEventListener("click", mostrarRegistro);
document.getElementById("listadoAlumnos").addEventListener("click", mostrarRegistro);
document.getElementById("notasMedias").addEventListener("click", mostrarRegistro);
document.getElementById("cerrarSesion").addEventListener("click", mostrarRegistro);
document.getElementById("btnRegistrarNotas").addEventListener("click", registrarNota);
document.getElementById("selAlumnoDetalle").addEventListener("change", mostrarDetalle);

ocultarTodo();
```

```
function ocultarTodo(){
    document.getElementById("formRegistrarAlumno").style.display = "none";
    document.getElementById("formRegistrarNotas").style.display = "none";
    document.getElementById("formDetalle").style.display = "none";
    document.getElementById("lNotasMedias").style.display = "none";
    document.getElementById("detalleAlumno").style.display = "none";
    document.getElementById("listado").style.display = "none";
}
```

```
/** @param {Event} event */
function mostrarRegistro(event){
    event.preventDefault();
    ocultarTodo();
    switch (event.target.id) {
        case "registroAlumnos":
            document.getElementById("formRegistrarAlumno").style.display="block";
            break;

        case "registroNotas":
            document.getElementById("formRegistrarNotas").style.display="block";
            break;

        case "listadoAlumnos":
            document.getElementById("listado").style.display="block";
            document.getElementById("formDetalle").style.display="block";
            document.getElementById("selAlumnoDetalle").value="";
            mostrarListado();
            break;

        case "notasMedias":
            document.getElementById("lNotasMedias").style.display="block";
            mostrarMedias();
            break;

        default:
            break;
    }
}
```

Menú: un solo manejador para todos los enlaces. Primero `ocultarTodo()` y luego `style.display = "block"` solo a la sección del enlace pulsado (`event.target.id`). `preventDefault()` evita que el `href="#"` haga nada. Se llama a `ocultarTodo()` también al cargar.

Registrar alumno

```
function registrarAlumno(){
    const nombre= document.querySelector("input[name=nombre]").value;
    const apellidos= document.querySelector("input[name=Apellidos]").value;
    const usuario= document.querySelector("input[name=usuario]").value;
    const contrasena= document.querySelector("input[name=contrasena]").value;
    const correo= document.querySelector("input[name=correo]").value;

    if(nombre=="" || apellidos== "") {
        alert("El nombre y los apellidos son obligatorios");
        return;
    }

    if(usuario==""){
        alert("El usuario no debe estar vacio");
        return;
    }

    if(validarContrasena(contrasena) == false){
        alert("Debe contener un numero, letra minuscula, letra mayuscula y tener una longitud
minima de 6.");
        return;
    }
    if( validarCorreo(correo) == false){
        alert("Debe contener un @, y un .");
        return;
    }

    const alumno= new Alumno(nombre, apellidos, usuario, contrasena, correo);
    alumnos.push(alumno);
    actualizarDesplegables();
    alert("Alumno registrado");
    document.getElementById("formRegistrarAlumno").reset();
}
```

Orden: leer cada input en su variable → validar con esas variables (cada una con `alert` + `return`) → `new Alumno(...)` → `push` → refrescar desplegables → aviso → `reset()`. `validarContrasena`, `validarCorreo` y `crearUsuario` son las del Ejercicio Formularios:

```
function crearUsuario() {
    const nombre=document.querySelector("input[name=nombre]").value;
    const apellido=document.querySelector("input[name=Apellidos]").value;

    let primerApellido=apellido.split(" ")[0];
    let segundoApellido=apellido.split(" ")[1];

    let nombreUsuario=nombre[0];

    for (let i = 0; i < primerApellido.length ; i++) {
        let letra = primerApellido[i];
        if(i<=2){
            nombreUsuario+=letra;
        }
    }

    for (let i = 0; i < segundoApellido.length ; i++) {
        let letra = segundoApellido[i];
        if(i<=2){
            nombreUsuario+=letra;
        }
    }
    nombreUsuario=nombreUsuario.toLowerCase();
    document.querySelector("input[name=usuario]").value=nombreUsuario;
}
```

```
function validarContrasena(contrasena) {
    let contieneNumero=false;
    const abecedarioMin='abcdefghijklmnopqrstuvwxyz';
    const abecedarioMayusculas = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
    const numerosCadena = '0123456789';
    let longitudCorrecta=false;
    let letraMin=false;
    let letraMay=false;
    let contrasenaValida=false;

    if(contrasena.length>=6) {
        longitudCorrecta=true;
    }
    for (let i = 0; i < contrasena.length; i++) {
        let letra  = contrasena[i];
        if(numerosCadena.includes(letra)){
            contieneNumero=true;
        }
        if(abecedarioMin.includes(letra)){
            letraMin=true;
        }
        if(abecedarioMayusculas.includes(letra)){
            letraMay=true;
        }
    }
    if(longitudCorrecta==true && contieneNumero==true && letraMin==true && letraMay==true) {
        contrasenaValida=true;
    }
    return contrasenaValida;
}
```

```
function validarCorreo(correo){
    let correoValido=false;

    if(correo.includes("@") && correo.includes(".")){
        correoValido=true;
    }
    return correoValido;
}
```

Rellenar los desplegables

```
function actualizarDesplegables(){
    const select=document.getElementById("selAlumno");
    select.innerHTML="";
    const detalle=document.getElementById("selAlumnoDetalle");
    detalle.innerHTML="";

    // Primera opción vacía: así el evento change salta al elegir CUALQUIER alumno
    const opcionVacia=document.createElement("option");
    opcionVacia.value="";
    opcionVacia.textContent="Selecciona un alumno";
    detalle.appendChild(opcionVacia);

    for (const alumno of alumnos) {
        const option = document.createElement("option");
        option.value=alumno.getUsuario();
        option.innerHTML=alumno.getNombre() + " " + alumno.getApellidos();
        select.appendChild(option);

        const option2=document.createElement("option");
        option2.value=alumno.getUsuario();
        option2.innerHTML=alumno.getNombre() + " " + alumno.getApellidos();
        detalle.appendChild(option2);
    }
}
```

Vaciar los selects y volver a crear todas las opciones desde el array. La opción vacía "Selecciona un alumno" evita la **trampa del `change`**: si el alumno ya está seleccionado (el único o el primero), elegirlo no dispara el evento.

Registrar nota

```
function registrarNota(){
    const usuario=document.getElementById("selAlumno").value;
    const asignatura=document.querySelector("select[name=selNota]").value;
    const nota=parseFloat(document.querySelector("input[name=nota]").value);

    if(isNaN(nota)){
        alert("No puede estar vacio");
        return;
    }
    if(nota<0 || nota>10) {
        alert("La nota debe estar entre 0 y 10");
        return;
    }
    for (const alumno of alumnos) {
        if(usuario==alumno.getUsuario()){
            const obNota= new Nota(asignatura,nota);
            const existe= alumno.altaNota(obNota);
            if(existe){
                alert("Esa asignatura ya tenia la nota guardada");
            } else {
                alert("Nota registrada");
            }
        }
    }

    document.getElementById("formRegistrarNotas").reset();
}
```

Validar primero que haya número (`isNaN`) y luego el rango. Buscar al alumno con un bucle + `if (usuario == alumno.getUsuario())`. `altaNota` ya guarda la nota: no se hace `push`; su resultado solo sirve para avisar.

Detalle de un alumno (evento change)

```
function mostrarDetalle(){
    const usuario=document.getElementById("selAlumnoDetalle").value;
    const capa = document.getElementById("detalleAlumno");
    capa.innerHTML="";

    // Si se vuelve a "Selecciona un alumno", se oculta el detalle
    if(usuario==""){
        capa.style.display="none";
        return;
    }

    for (const alumno of alumnos) {
        if(usuario==alumno.getUsuario()){
            capa.style.display="block";

            if(alumno.getLongitudArrayNotas()==0){
                capa.textContent="Este alumno no tiene notas registradas";
                return;
            }

            const detalle=document.createElement("table");
            detalle.className="table table-striped";

            for (let i = 0; i < alumno.getLongitudArrayNotas(); i++) {
                let notaAsignatura = alumno.getAsignaturaYNota(i);
                let partirNotaAsignatura=notaAsignatura.split(":");
                let asignatura=partirNotaAsignatura[0];
                let nota=partirNotaAsignatura[1];
                const fila=document.createElement("tr");
                const columnaAsig=document.createElement("td");
                const columnaNota=document.createElement("td");
                columnaAsig.textContent=asignatura;
                columnaNota.textContent=nota;
                fila.appendChild(columnaAsig);
                fila.appendChild(columnaNota);
                detalle.appendChild(fila);
            }

            capa.appendChild(detalle);
        }
    }
}
```

`capa` es el div que ya existe (mostrar con `"block"`, sin `;` dentro de las comillas, y vaciar); `detalle` es la tabla nueva. `getAsignaturaYNota(i).split(":")` separa asignatura y nota.

Notas medias

```
function mostrarMedias(){
    const notas=document.getElementById("lNotasMedias");
    notas.innerHTML="";

    if(alumnos.length==0){
        notas.textContent="No hay alumnos registrados";
        return;
    }

    const tableNotaMedia=document.createElement("table");
    tableNotaMedia.className="table table-striped";

    for (const alumno of alumnos) {
        // 1. Para CADA alumno: la suma empieza en 0
        let sumaNotas=0;
        const numNotas=alumno.getLongitudArrayNotas();

        // 2. Bucle de notas: SOLO sumar
        for (let i = 0; i < numNotas; i++) {
            sumaNotas += alumno.getNota(i);
        }

        // 3. Ya fuera del bucle de notas: la media de este alumno
        let media;
        if(numNotas==0){
            media="Sin notas";
        } else {
            media=(sumaNotas/numNotas).toFixed(2);
        }

        // 4. UNA fila por alumno
        const fila=document.createElement("tr");
        const columnaNombre=document.createElement("td");
        const columnaMedia=document.createElement("td");
        columnaNombre.textContent=alumno.getNombre() + " " + alumno.getApellidos();
        columnaMedia.textContent=media;
        fila.appendChild(columnaNombre);
        fila.appendChild(columnaMedia);
        tableNotaMedia.appendChild(fila);
    }

    notas.appendChild(tableNotaMedia);
}
```

> **Qué va en cada bucle:** el de **alumnos** pone la suma a 0, calcula la media y crea **una** fila; el de **notas** solo suma. Si la fila se crea en el de dentro sale una por nota (y fuera no existe). `toFixed(2)` deja dos decimales; si no hay notas, "Sin notas" (evita dividir entre 0).

Listado con botón Eliminar (review.js)

En el HTML se añade un contenedor: `<div id="listado" style="display: none; width: 1000px; margin: 30px auto"></div>`, que también se oculta en `ocultarTodo()` y se muestra en el `case "listadoAlumnos"` llamando a `mostrarListado()`.

```
// Tabla Bootstrap con nombre, apellidos, usuario y correo, y un botón Eliminar por fila
function mostrarListado(){
    const capa=document.getElementById("listado");
    capa.innerHTML="";

    if(alumnos.length==0){
        capa.textContent="No hay alumnos registrados";
        return;
    }

    const tabla=document.createElement("table");
    tabla.className="table table-striped";

    // Fila de cabecera con <th>
    const cabecera=document.createElement("tr");
    const titulos=["Nombre", "Apellidos", "Usuario", "Correo", ""];
    for (const titulo of titulos) {
        const th=document.createElement("th");
        th.textContent=titulo;
        cabecera.appendChild(th);
    }
    tabla.appendChild(cabecera);

    // Una fila por alumno
    for (const alumno of alumnos) {
        const fila=document.createElement("tr");

        const datos=[alumno.getNombre(), alumno.getApellidos(), alumno.getUsuario(),
alumno.getCorreo()];
        for (const dato of datos) {
            const td=document.createElement("td");
            td.textContent=dato;
            fila.appendChild(td);
        }

        // Celda con el botón Eliminar: guarda el usuario en un atributo data-usuario
        const tdBoton=document.createElement("td");
        const boton=document.createElement("button");
        boton.type="button";
        boton.className="btn btn-danger btn-sm";
        boton.textContent="Eliminar";
        boton.dataset.usuario=alumno.getUsuario();
        boton.addEventListener("click", eliminarAlumno);
        tdBoton.appendChild(boton);
        fila.appendChild(tdBoton);

        tabla.appendChild(fila);
    }

    capa.appendChild(tabla);
}

/** @param {Event} event */
function eliminarAlumno(event){
    const usuario=event.target.dataset.usuario;

    // Buscar la posición del alumno en el array
    let posicion=-1;
    for (let i = 0; i < alumnos.length; i++) {
        if(alumnos[i].getUsuario()==usuario){
            posicion=i;
        }
    }
    if(posicion==-1){
```

```
        return;
    }

    const alumno=alumnos[posicion];
    const nombreCompleto=alumno.getNombre() + " " + alumno.getApellidos();

    // confirm devuelve true si se pulsa Aceptar y false si se pulsa Cancelar
    if(!confirm("¿Está seguro de que desea eliminar al alumno " + nombreCompleto + "?")){
        return;
    }

    alumnos.splice(posicion, 1);          // quitarlo del array
    actualizarDesplegables();             // que desaparezca también de los selects
    document.getElementById("detalleAlumno").style.display="none";
    mostrarListado();                     // volver a pintar la tabla sin él
    alert("El alumno ha sido eliminado con éxito");
}
```

Cómo funciona:

1. **Cabecera y filas con arrays:** un array de títulos para los `th` y, por alumno, un array con sus datos recorrido con un bucle. Menos líneas repetidas que crear cada `td` a mano.
2. **Atributos `data-`** (diapositiva 19): cada botón guarda el usuario de su alumno con `boton.dataset.usuario = …` (en el HTML: `data-usuario="rmorpac"`). Al pulsar, `event.target.dataset.usuario` dice a quién borrar, aunque todos los botones llamen a la misma función.
3. **Evento al crear el botón:** los botones no existen al cargar, así que el `addEventListener` se pone dentro del bucle, justo al crearlos.
4. **`confirm(texto)`** muestra Aceptar/Cancelar y devuelve `true`/`false`; con `! + return` se sale si se cancela.
5. **`alumnos.splice(posicion, 1)`** quita 1 elemento en esa posición del array.
6. **Volver a pintar** todo lo que depende del array: desplegables y tabla.

12. Ejercicio Final «TiendaDAW» resuelto (examen de prueba)

Carpeta `Ejercicio Final DOM`. Clase `Producto` dada (`#codigo, #nombre, #categoria, #precio, #stock`, getters, `vender(cantidad)` → **true/false**, `reponer(cantidad)`, `getValorTotal()`, `toString()`). Scripts en el `<head>` → **DOMContentLoaded obligatorio**.

Estructura del archivo y dónde va cada cosa

```
document.addEventListener("DOMContentLoaded", iniciar);   // fuera: SOLO esto (y las variables)

function iniciar() {
    // 1. TODOS los listeners (aquí el HTML ya existe)
    // 2. Llamadas iniciales: cargarDatos(); mostrarSeccion("inventario"); pintarTabla();
}

function cargarDatos() { ... }     // las funciones, fuera y abajo
...
```

Dónde	Qué va
Fuera, arriba	Variables (`let productos = []`...) y `document.addEventListener("DOMContentLoaded", iniciar)`
Dentro de `iniciar()`	**Todo** lo que hace `getElementById/querySelector` al arrancar: listeners y llamadas iniciales
Fuera, abajo	Las funciones (definirlas no ejecuta nada; se ejecutan cuando alguien las llama)

```
function iniciar() {
    // 1. Listeners (aquí el HTML ya existe)
    document.getElementById("menu").addEventListener("click", mostrarMenu);
// apartado 2
    document.querySelector("select[name=categoria]").addEventListener("change", codCategoria);
// apartado 3a
    document.getElementById("formAlta").addEventListener("submit", altaProducto);
// apartado 3b
    document.getElementById("cuerpoTabla").addEventListener("click", accionTabla);
// apartado 5
    document.getElementById("buscador").addEventListener("input", pintarTabla);
// apartado 6
    document.getElementById("buscador").addEventListener("keydown", teclaBuscador);
// apartado 6
    document.getElementById("filtroCategoria").addEventListener("change", pintarTabla);
// apartado 6
    document.getElementById("btnOrdenar").addEventListener("click", cambiarOrden);
// apartado 6

    // 2. Llamadas iniciales
    cargarDatos();
    crearOferta();                 // Extra A
    mostrarSeccion("inventario");
    pintarTabla();
}
```

Orden del navegador: 1) lee el `<head>` y ejecuta lo suelto del script (el body aún no existe → `null`) · 2) lee el body · 3) salta `DOMContentLoaded` → `iniciar()`. Un listener suelto arriba da `Cannot read properties of null` y **para todo el script**.

Ap. 1 · Instanciar la clase desde datos normales

```
function cargarDatos() {
    for (const producto of datosIniciales) {
        const prod = new Producto(producto.codigo, producto.nombre, producto.categoria,
producto.precio, producto.stock);
        productos.push(prod);
    }
}
```

Tres nombres distintos: `producto` (objeto normal del for) · `prod` (el `new`) · `productos` (el array). Comprobar con `console.log(productos)` → `Producto {}` (vacío porque los `#` son privados).

Ap. 2 · Mostrar secciones y menú (delegación + data-seccion)

```
function mostrarSeccion(id) {
    const secciones = document.querySelectorAll(".seccion");
    for (const seccion of secciones) {
        seccion.style.display = "none";
    }
    document.getElementById(id).style.display = "block";
}
```

```
function mostrarMenu(event) {
    event.preventDefault();                          // el <a href="#"> no salta arriba
    if (event.target.tagName == "A") {               // solo si se ha pulsado un enlace
        const mostrar = event.target.dataset.seccion; // "alta" / "inventario" / "resumen"
        mostrarSeccion(mostrar);

        if (mostrar == "resumen") {                  // la sección resumen hace algo EXTRA
            pintarResumen();
        }
    }
}
```

El `data-seccion` del enlace vale lo mismo que el `id` de su sección → **no hace falta switch**: se lee y se pasa tal cual. `tagName` = etiqueta (`"A"`), `data-` = atributo. Una función que se llama y aún no existe → créala vacía (`pintarResumen`) para no tener `is not defined`.

Ap. 3a · Código automático (substring + filter + ceros)

```
function codCategoria() {
    const categoria = document.querySelector("select[name=categoria]").value;
    const inputCodigo = document.querySelector("input[name=codigo]");

    // 1. Si vuelve a "-- Elige categoría --" (value vacío) → código vacío y salimos
    if (categoria == "") {
        inputCodigo.value = "";
        return;
    }

    // 2. Las 3 primeras letras en mayúscula: "Electrónica" → "ELE"
    const letras = categoria.substring(0, 3).toUpperCase();

    // 3. Cuántos productos hay YA de esa categoría, + 1
    //    filter devuelve un array nuevo solo con los que cumplen la condición → .length =
cuántos son
    const numero = productos.filter(p => p.getCategoria() == categoria).length + 1;

    // 4. El número con 3 cifras (sin padStart): 3 → "003", 12 → "012", 125 → "125"
    let cifras;
    if (numero < 10) {
        cifras = "00" + numero;
    } else if (numero < 100) {
        cifras = "0" + numero;
    } else {
        cifras = "" + numero;      // "" + número → lo convierte en texto
    }

    // 5. Juntarlo y meterlo en el input
    inputCodigo.value = letras + "-" + cifras;     // "ELE-003"
}
```

`substring(0, 3)`: el 2º número **no se incluye** → 3 letras. `filter(...).length` = cuántos cumplen. Ceros sin `padStart`: `if` < 10 → `"00"`, < 100 → `"0"`. Con `padStart`: `String(n).padStart(3, "0")`.

Ap. 3b · Alta con validaciones (submit, errores juntos, some, charAt)

```
function altaProducto(event) {
    event.preventDefault();       // el submit recargaría la página y se perdería todo

    // 1. Leer los campos (el form tiene name en cada input/select)
    const categoria = document.querySelector("select[name=categoria]").value;
    const codigo = document.querySelector("input[name=codigo]").value;
    let nombre = document.querySelector("input[name=nombre]").value.trim();   // trim: quita
espacios de los lados
    const textoPrecio = document.querySelector("input[name=precio]").value;
    const textoStock = document.querySelector("input[name=stock]").value;
    const precio = parseFloat(textoPrecio);    // "45.5" → 45.5   |  "" o "abc" → NaN
    const stock = Number(textoStock);          // "3" → 3   |  "3.5" → 3.5  |  "abc" → NaN

    // 2. Validar: se GUARDAN todos los errores en un array y se enseñan juntos al final
    const errores = [];

    if (categoria == "") {
        errores.push("Debes elegir una categoría");
    }

    if (nombre.length < 3) {
        errores.push("El nombre debe tener al menos 3 caracteres");
    }

    // some → true si ALGÚN producto ya tiene ese nombre (comparando los dos en minúsculas)
    const repetido = productos.some(p => p.getNombre().toLowerCase() == nombre.toLowerCase());
    if (repetido) {
        errores.push("Ya existe un producto con ese nombre");
    }

    if (isNaN(precio) || precio <= 0) {
        errores.push("El precio debe ser un número mayor que 0");
    }

    // Number.isInteger(3) → true | Number.isInteger(3.5) → false | Number.isInteger(NaN) →
false
    if (textoStock == "" || !Number.isInteger(stock) || stock < 0) {
        errores.push("El stock debe ser un número entero mayor o igual que 0");
    }

    const mensajes = document.getElementById("mensajes");

    // 3. Si hay errores → lista roja y salimos
    if (errores.length > 0) {
        let lista = "";
        for (const error of errores) {
            lista += `<li>${error}</li>`;
        }
        mensajes.innerHTML = `<div class="alert alert-danger"><ul>${lista}</ul></div>`;
        return;
    }

    // 4. Formatear el nombre: "tECLADO" → "T" + "eclado" → "Teclado"
    nombre = nombre.charAt(0).toUpperCase() + nombre.slice(1).toLowerCase();

    // 5. Crear el objeto de la clase y guardarlo
    const nuevo = new Producto(codigo, nombre, categoria, precio, stock);
    productos.push(nuevo);

    // 6. Mensaje verde, limpiar el formulario y repintar la tabla
    mensajes.innerHTML = `<div class="alert alert-success">Producto ${codigo} añadido</div>`;
    document.getElementById("formAlta").reset();     // reset vacía también el input del código
```

```
    pintarTabla();
}
```

`submit` va en el **form** + `preventDefault()`. Errores en un **array** y se pintan todos al final. `some` → ¿ya existe? (comparar en `toLowerCase()`). `Number.isInteger` para enteros. Primera letra mayúscula: `charAt(0).toUpperCase() + slice(1).toLowerCase()`.

Ap. 4 y 6 · Pintar la tabla (template literals, filtros y orden)

```
function pintarTabla() {
    const cuerpo = document.getElementById("cuerpoTabla");
    cuerpo.innerHTML = "";        // 1. vaciar antes del bucle

    // 2. Leer los filtros (apartado 6). Se leen SIEMPRE: así se mantienen al vender, borrar...
    const texto = document.getElementById("buscador").value.trim().toLowerCase();
    const categoria = document.getElementById("filtroCategoria").value;

    // 3. filter → array NUEVO solo con los que cumplen las dos condiciones
    const lista = productos.filter(p =>
        p.getNombre().toLowerCase().includes(texto) &&             // contiene el texto ("" lo
contienen todos)
        (categoria == "Todas" || p.getCategoria() == categoria)    // "Todas" o esa categoría
    );

    // 4. Ordenar la COPIA (lista), nunca "productos" → el array original no se desordena
    if (ordenAscendente) {
        lista.sort((a, b) => a.getPrecio() - b.getPrecio());   // menor a mayor
    } else {
        lista.sort((a, b) => b.getPrecio() - a.getPrecio());   // mayor a menor
    }

    // 5. Nada que enseñar → una sola fila que ocupa las 6 columnas
    if (lista.length == 0) {
        cuerpo.innerHTML = `<tr><td colspan="6">No hay productos</td></tr>`;
        return;
    }

    // 6. Una fila por producto con template literal
    for (const p of lista) {
        // Color de la fila según el stock
        let claseFila = "";
        if (p.getStock() == 0) {
            claseFila = "table-danger";       // rojo
        } else if (p.getStock() < 3) {
            claseFila = "table-warning";      // amarillo
        }

        // Precio: normal, o precio final + OFERTA si es un ProductoOferta (Extra A)
        let textoPrecio = p.getPrecio().toFixed(2) + " €";
        if (p instanceof ProductoOferta) {
            textoPrecio = `<s>${p.getPrecio().toFixed(2)} €</s>
${p.getPrecioFinal().toFixed(2)} € <b>OFERTA</b>`;
        }

        // Cada botón lleva DOS data-: qué hace (accion) y de qué producto es (codigo)
        cuerpo.innerHTML += `
            <tr class="${claseFila}">
                <td>${p.getCodigo()}</td>
                <td>${p.getNombre()}</td>
                <td>${p.getCategoria()}</td>
                <td>${textoPrecio}</td>
                <td>${p.getStock()}</td>
                <td>
                    <button class="btn btn-sm btn-success" data-accion="vender" data-
codigo="${p.getCodigo()}">Vender</button>
                    <button class="btn btn-sm btn-primary" data-accion="reponer" data-
codigo="${p.getCodigo()}">Reponer +5</button>
                    <button class="btn btn-sm btn-danger" data-accion="eliminar" data-
codigo="${p.getCodigo()}">Eliminar</button>
                </td>
            </tr>`;
```

```
    }
}
```

`innerHTML = ""` antes del bucle y `+=` dentro. `pintarTabla` lee **siempre** el buscador y el select → los filtros se mantienen al vender o eliminar. `sort` sobre la lista filtrada (array nuevo), **nunca sobre `productos`**. Cada botón lleva `data-accion` + `data-codigo`.

Ap. 5 · Botones de la tabla: delegación con data-accion y data-codigo

```
function accionTabla(event) {
    const accion = event.target.dataset.accion;   // "vender" / "reponer" / "eliminar"
    const codigo = event.target.dataset.codigo;   // "ELE-001"...

    // Si se ha pulsado una celda (no un botón) → no tiene data-accion → undefined → nada
    if (accion == undefined) {
        return;
    }

    // find → el PRODUCTO (objeto) cuyo código coincide
    const producto = productos.find(p => p.getCodigo() == codigo);

    if (accion == "vender") {
        const vendido = producto.vender(1);       // la clase devuelve true / false
        if (vendido) {
            const hora = new Date().toLocaleTimeString();    // "19:24:45"
            ventas.push(`${hora} - 1 x ${producto.getNombre()} -
${producto.getPrecio().toFixed(2)} €`);
        } else {
            alert("No hay stock de " + producto.getNombre());
        }
    } else if (accion == "reponer") {
        producto.reponer(5);
    } else if (accion == "eliminar") {
        if (confirm("¿Eliminar este producto?\n" + producto.toString())) {
            // findIndex → la POSICIÓN; splice(posición, 1) → borra 1 elemento desde ahí
            const posicion = productos.findIndex(p => p.getCodigo() == codigo);
            productos.splice(posicion, 1);
        }
    }

    pintarTabla();     // después de cualquier botón
}
```

Un **solo** listener en el `<tbody>` (las filas se crean después con `innerHTML`, por eso delegación). `dataset.accion` dice qué botón, `dataset.codigo` de qué producto. Pulsar una celda → `undefined` → `return`.

★ VENDER: find + método de la clase (true / false)

```
// find → el OBJETO producto (para usar sus métodos)
const producto = productos.find(p => p.getCodigo() == codigo);
// la clase resta el stock y devuelve true (vendido) o false (sin stock)
const vendido = producto.vender(1);
if (vendido) {
    const hora = new Date().toLocaleTimeString();     // "19:24:45"
    ventas.push(`${hora} - 1 x ${producto.getNombre()} - ${producto.getPrecio().toFixed(2)}
€`);
} else {
    alert("No hay stock de " + producto.getNombre());
}
pintarTabla();
```

No restes el stock a mano: los atributos son privados (`#stock`) y el método ya lo hace. Usa lo que **devuelve** el método para decidir.

★ REPONER: método de la clase

```
const producto = productos.find(p => p.getCodigo() == codigo);
producto.reponer(5);          // suma 5 al stock dentro de la clase
pintarTabla();
```

★ ELIMINAR: confirm + findIndex + splice

```
// confirm → Aceptar = true, Cancelar = false
if (confirm("¿Eliminar este producto?\n" + producto.toString())) {
    // findIndex → la POSICIÓN (un número), no el objeto
    const posicion = productos.findIndex(p => p.getCodigo() == codigo);
    // splice(posición, 1) → borra 1 elemento desde esa posición
    productos.splice(posicion, 1);
}
pintarTabla();     // repintar SIEMPRE después
```

Método	Devuelve	Para qué
`find(cond)`	el **objeto** (o `undefined`)	usar sus métodos: `vender`, `reponer`, `toString`
`findIndex(cond)`	la **posición** (o `-1`)	borrar con `splice(posicion, 1)`
`filter(cond)`	array **nuevo** con los que cumplen	borrar "todos menos": `productos = productos.filter(p => p.getCodigo() != codigo)`
`splice(pos, n)`	modifica el array original	quitar `n` elementos desde `pos`

`splice` necesita la **posición**, no el objeto ni el código: `productos.splice("ELE-001", 1)` no funciona. Si `findIndex` da `-1` (no existe), `splice(-1, 1)` borraría el **último**.

Ap. 6 · Tecla Escape y botón ordenar

```
function teclaBuscador(event) {
    if (event.key == "Escape") {
        event.target.value = "";     // vaciar el buscador
        pintarTabla();
    }
}
```

```
function cambiarOrden() {
    ordenAscendente = !ordenAscendente;      // true ↔ false
    const boton = document.getElementById("btnOrdenar");
    if (ordenAscendente) {
        boton.textContent = "Ordenar por precio ↑";
    } else {
        boton.textContent = "Ordenar por precio ↓";
    }
    pintarTabla();
}
```

El `input` del buscador y el `change` del select llaman directamente a `pintarTabla` (el filtro está dentro). `keydown` = evento, `event.key` = la tecla. `!variable` invierte true ↔ false.

Ap. 7 · Resumen: sumas, el máximo y join

```
function pintarResumen() {
    let unidades = 0;
    let valor = 0;
    let masCaro = null;
    const agotados = [];

    // Un solo bucle para calcularlo todo
    for (const p of productos) {
        unidades += p.getStock();
        valor += p.getValorTotal();

        if (masCaro == null || p.getPrecio() > masCaro.getPrecio()) {
            masCaro = p;
        }
        if (p.getStock() == 0) {
            agotados.push(p.getNombre());
        }
    }

    let textoMasCaro = "-";
    if (masCaro != null) {
        textoMasCaro = masCaro.getNombre();
    }

    let textoAgotados = "Ninguno";
    if (agotados.length > 0) {
        textoAgotados = agotados.join(", ");    // ["A", "B"] → "A, B"
    }

    document.getElementById("datosResumen").innerHTML = `
        <p><b>Productos:</b> ${productos.length}</p>
        <p><b>Unidades en stock:</b> ${unidades}</p>
        <p><b>Valor del inventario:</b> ${valor.toFixed(2)} €</p>
        <p><b>Producto más caro:</b> ${textoMasCaro}</p>
        <p><b>Agotados:</b> ${textoAgotados}</p>`;

    // Historial: un <li> por venta
    const historial = document.getElementById("historial");
    historial.innerHTML = "";
    if (ventas.length == 0) {
        historial.innerHTML = "<li>Sin ventas</li>";
    } else {
        for (const venta of ventas) {
            historial.innerHTML += `<li>${venta}</li>`;
        }
    }
}
```

Un bucle con acumuladores (`+=`). Máximo: guardar el primero y cambiarlo si otro es mayor. `join(", ")` junta un array en texto.

Extra A · Herencia: extends, super e instanceof

```
class ProductoOferta extends Producto {
    #descuento;     // porcentaje: 20 = 20 %

    constructor(codigo, nombre, categoria, precio, stock, descuento) {
        super(codigo, nombre, categoria, precio, stock);   // los 5 primeros los guarda la
clase padre
        this.#descuento = descuento;
    }

    getDescuento() {
        return this.#descuento;
    }

    getPrecioFinal() {
        return this.getPrecio() - this.getPrecio() * this.#descuento / 100;   // 60 € con 20 %
→ 48 €
    }
}

function crearOferta() {
    const oferta = new ProductoOferta("DEP-002", "Pala de pádel", "Deporte", 60, 3, 20);
    productos.push(oferta);     // es un Producto también (hereda) → la tabla, vender, etc.
funcionan igual
}
```

`extends` hereda todo; en el `constructor` lo primero es `super(...)` con los datos del padre. Los `#` del padre no se ven desde el hijo → se usan sus getters (`this.getPrecio()`). `p instanceof ProductoOferta` → `true`/`false`; un ProductoOferta también es `instanceof Producto`.

Fallos que tuve en este ejercicio (y su error en consola)

Qué hice	Qué pasa / mensaje
Función con el mismo nombre que el array (`function datosIniciales`)	`Identifier 'datosIniciales' has already been declared`
`push` del objeto normal en vez de un `new Producto`	`p.getNombre is not a function (más adelante)`
Líneas sueltas `producto.codigo;`	`no hace nada: el valor se tira`
`Producto(...)` sin `new`	`Class constructor Producto cannot be invoked without 'new'`
`const producto = new ...` dentro de `for (const producto ...)`	`Cannot access 'producto' before initialization`
`const productos = new ...` (tapa el array)	`productos.push is not a function`
Función escrita pero nunca llamada	`no pasa nada, ni el console.log`
`console.log` dentro del bucle	`se imprime N veces`
`addEventListener("DOMContentLoaded")` en un elemento / listener suelto con script en head	`Cannot read properties of null (reading 'addEventListener')`
`const id = "inventario"` dentro de `function mostrarSeccion(id)`	`Identifier 'id' has already been declared`
`querySelector` para varias	`solo coge la primera`
`"section[class=.seccion]"` / `".section"`	`lista vacía → ".seccion"`
`getElementById` sin `document.`	`getElementById is not defined`

`.display` sin `.style`	`no hace nada`
`tagName == "DATA-SECCION"`	`nunca se cumple: tagName es la etiqueta`
`event.target.id` en los enlaces	`"": el id lo tienen las secciones`
`mostrarSeccion()` sin argumento	`Cannot read properties of null (reading 'style')`
`substring(0, 2)` para 3 letras	`"EL": el 2º número no se incluye`
Llamar a `pintarResumen()` sin crearla	`pintarResumen is not defined`
Error `favicon.ico 404`	`no es tuyo: ignorar`

> **En el examen:** antes de escribir JS, abre el `index.html` y apunta `id`, `name`, clases y `data-`. El enunciado no los dice todos. Si un apartado necesita una función que aún no has hecho, créala vacía y sigue.

13. Errores típicos (revisar antes de entregar)

- `const x = elem.addEventListener(...)` → guarda `undefined`.
- `getElementsBy…` usado como un solo elemento (falta `[0]` o un bucle).
- Olvidar `.value` en un input, o usarlo en un `p`/`td`/`div`/`form`.
- Leer inputs fuera de la función del evento (al cargar están vacíos).
- `.length` de un texto como si fuera el número escrito.
- `appendChild` al revés (`celda.appendChild(fila)`) o con algo que no es un elemento.
- `createElement("<table>")` en vez de `createElement("table")`; `createElement` sin `document.`
- Crear elementos que ya existen en el HTML (botones, selects…): se cogen, no se crean.
- `setAttribute("texto", "style=…")`: elemento delante del punto, atributo primero, valor después.
- `if (elemento)` para saber si se ha pulsado/elegido: siempre es verdadero si existe. Usar `event.target`, `.value`, `.checked`, `.selected`.
- `=` en vez de `==`/`===` en un `if`; dos `if` seguidos que se deshacen (usar `else`).
- Asignaciones al revés: **donde guardo = lo que guardo**.
- Línea en el bloque equivocado (dentro/fuera de la función o del bucle).
- Evento en el elemento equivocado: `submit` va en el form; el clic en lo que se pulsa, no en lo que cambia; en toda la página → `document`.
- Confundir el nombre del evento (`"keydown"`) con una propiedad (`event.key`).
- Recorrer una lista viva mientras se mueven elementos (se salta alguno).
- Variables sin `let`/`const`.

14. De las diapositivas, aún sin usar

Diap.	Contenido
9–10	Recorrer el DOM: `childNodes`, `firstChild`, `lastChild`, `parentNode`, hermanos
17	`outerHTML`: el HTML del elemento incluido él mismo
22–23	`prepend`, `before`, `after`, `replaceWith` · `insertAdjacentHTML("beforeend", html)`
26	`style.cssText` · `style.removeProperty("background")`
29	`elem.onclick = funcion` (solo admite uno; por eso mejor `addEventListener`)
36	Script al final del `body` (o en el head con `defer`) para que el HTML exista al ejecutarse

15. Avance Unidad 5: fetch (no entra en el examen de DOM)

Cuándo se usa: cuando la página necesita **datos de un servidor sin recargarse**. En los ejercicios de DOM los datos vivían solo en el navegador (el array `alumnos`): al recargar se pierden. Con `fetch` la página pide o envía datos en segundo plano y luego los pinta con el DOM de siempre.

Situación	Petición
Al abrir la página, cargar el listado guardado	**GET** (pedir datos)
Al pulsar "Registrar", guardarlo en la base de datos	**POST** (crear)
Modificar / borrar en el servidor	**PUT** / **DELETE**
Buscador que muestra resultados mientras escribes; tiempo, precios, resultados...	GET a una API

```
// Con .then
fetch("alumnos.json")                          // 1. pedir
    .then(respuesta => respuesta.json())       // 2. convertir a array/objeto de JS
    .then(datos => { /* 3. usar los datos: pintar */ })
    .catch(error => alert("No se han podido cargar los datos"));

// Con async/await (se lee de arriba abajo)
async function cargar() {
    try {
        const respuesta = await fetch("alumnos.json");
        const datos = await respuesta.json();
        // usar datos
    } catch (error) {
        alert("No se han podido cargar los datos");
    }
}
```

Es asíncrono: `fetch` devuelve una **promesa**; los datos solo existen dentro del `.then` o después del `await`.
Live Server: como las cookies, no funciona abriendo el HTML con doble clic (`file://`).

Ejemplos con ejercicios hechos

DOM 4 (colores): el array sale de `colores.json` (`["#ff0000", "#00ff00", …]`); el resto, igual.

```
async function colorCelda() {
    const respuesta = await fetch("colores.json");
    const colores = await respuesta.json();      // el mismo array que estaba escrito a mano
    // ... a partir de aquí, el código de la tabla 3×3 de siempre
}
```

Eventos 4 (provincias): la lista de la izquierda se rellena al abrir desde `provincias.json` (`[{ "codigo": "SE", "nombre": "Sevilla" }, …]`).

```
document.addEventListener("DOMContentLoaded", cargarProvincias);
async function cargarProvincias() {
    const provincias = await (await fetch("provincias.json")).json();
    const select = document.getElementById("lstProvincias");
    for (const p of provincias) {                // agregarProvincia dentro de un bucle
        const opt = document.createElement("option");
        opt.value = p.codigo;
        opt.textContent = p.nombre;
        select.appendChild(opt);
    }
}
```

Ejercicio Alumnos: cargar alumnos guardados desde `alumnos.json` y **convertirlos en objetos de la clase**.

```
document.addEventListener("DOMContentLoaded", cargarAlumnos);
async function cargarAlumnos() {
    const datos = await (await fetch("alumnos.json")).json();   // objetos NORMALES { nombre, …
}
    for (const d of datos) {
        alumnos.push(new Alumno(d.nombre, d.apellidos, d.usuario, d.contrasena, d.correo));
    }
    actualizarDesplegables();
}
```

> Lo que llega del JSON son **objetos literales** (`d.nombre`), sin `getNombre()`: hay que **instanciar** la clase con cada uno. Para guardar uno nuevo en el servidor se haría un `fetch` con **POST**, que necesita un servidor real (PHP + base de datos, final de la Unidad 5).